TRAIN SIM CLASSIC · VEHICLE WORKFLOW GUIDE

Part 0

Start Here: the whole pipeline on one page

How a Blender model becomes a drivable locomotive or a hauled wagon in Train Sim Classic, using Blender 3D + BRIAGE, Blueprint Editor 2, Lua and the game itself, and where the friction is.

The guide set

Part	Title	What you get from it
0	Start Here	The pipeline map, folder layout, conventions, glossary.
1	What a TSC Vehicle Actually Is	The game's own view of a loco/wagon: blueprints, components, coordinate systems, the control-value bus, the physics chain.
2	Modelling in Blender for TSC	Scale, axes, origins, the LOD naming scheme, bogies and wheels, materials/shaders, textures, IGS export with BRIAGE.
3	Animations and IA Export	Pantographs, wipers, doors and cab levers; one .ia per animation; frame rules; linking to controls.
4	Blueprint Editor 2	Building the engine, wagon, bogie, simulation and cab blueprints; every important field; unit traps.
5	Lua Scripting	Script lifecycle, the Call/SysCall API, patterns for custom features (with a trolley-pole example).
6	Build, Test and Ship	Export/compile, preview, scenario testing, packaging and a symptom-to-fix troubleshooting table.
7	Removing Workflow Friction	The Blender TSC Toolkit add-on and the cross-check tool shipped with this guide, and further script ideas.

Sources: Kuju/DTG sample vehicles (Sample Electric Engine, Example Steam Engine) supplied with this project, their Blender scene reads and raw geometry exports, the BRIAGE user manual v2.5.786, the community TS2015 scripting reference, and the project's own design specs. Where a statement is checked against those files it is marked **verified**; where it must be confirmed in your own tools it is marked **check**.

The pipeline at a glance

Every Train Sim Classic (TSC) vehicle goes through the same five stages. Blender makes the *shape*, BRIAGE converts it into the game's intermediate formats, Blueprint Editor 2 (BPE2) describes *what the shape is* (mass, bogies, controls, physics, scripts), the export step compiles everything, and the game loads the compiled result.

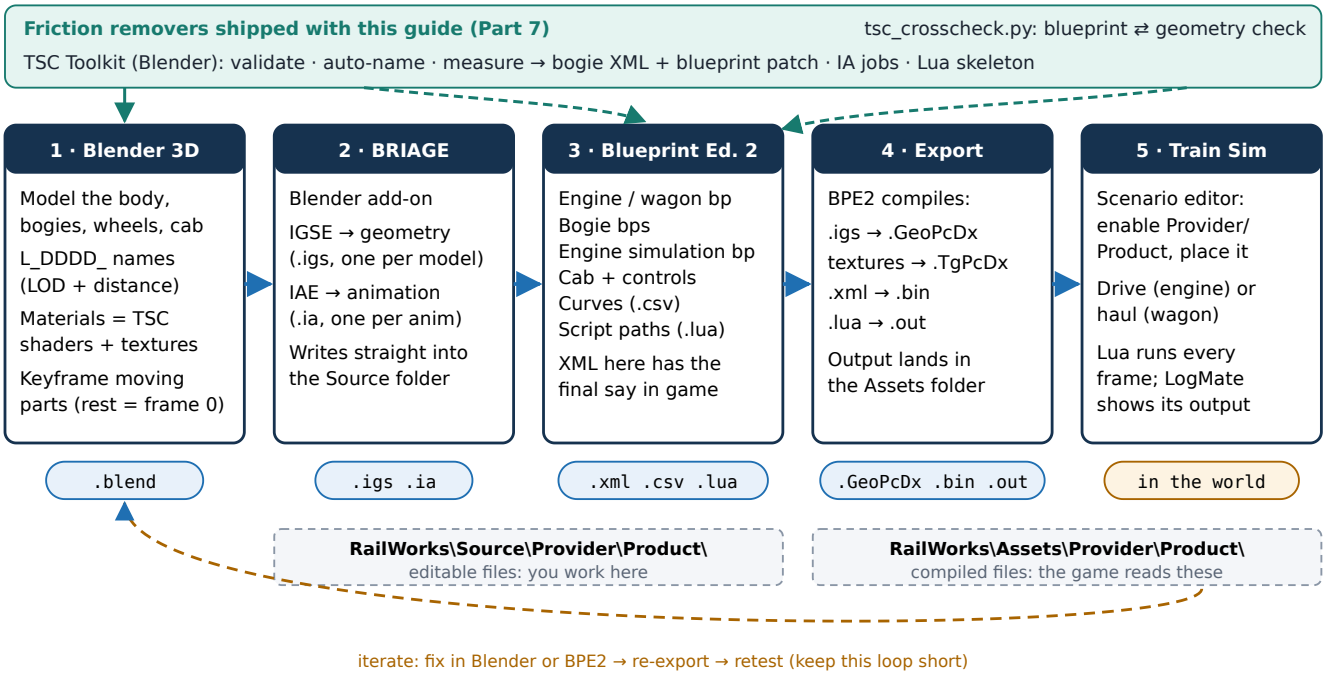


Figure 0.1 · The five stages and what each produces. The teal lane is the tooling from Part 7 that removes the manual copy-paste steps (measuring bogie pivots, re-typing coupling positions, checking node names). Everything left of the Export box lives in `Source`; everything the game loads lives in `Assets`.

Where the friction actually comes from

Converting a model is rarely hard because of one big problem. It is slow because of many small *disagreements* between three descriptions of the same vehicle: the Blender scene, the exported geometry, and the XML blueprints. The sample electric locomotive supplied with this project shows every one of them:

Disagreement	Example found in the Kuju sample electric	Fixed by
Bogie pivot vs mesh	Front bogie modelled at 5.583 m, blueprint pivots it at 6.25 m. The blueprint wins, so the bogie turns about a point 0.67 m ahead of where it is drawn. **verified**	Toolkit “Measure + Patch”
Wheel radius vs wheel height	Wheel centres at 0.525 m above the rail, radius set to 0.55 m. **verified**	Validate + Measure
Blueprint names a node the mesh lacks	`EngineStart` clicks node `start` and the pantograph lamp uses `hs_aus_lit`; neither exists in the cab geometry. **verified**	`tsc_crosscheck.py`

Disagreement	Example found in the Kuju sample electric	Fixed by
Collision box vs model	Box is 18 m long and 4.0 m high; the model is 19.12 m long and 4.53 m high. verified	Measure + Patch
Spawn defaults	`TrainBrakeControl` defaults to 0.7, so the loco spawns with its brakes 70% applied. verified	Cross-check lint
Stray data	An extra Blender-only material slot `f1` on the body; leftover “Class 47” names in the cab blueprint. verified	Validate

The guide's strategy is simple: **make Blender the single source of truth for anything geometric**, generate or patch the blueprint fields from it, and **check the links automatically** (node names, animation IDs, control names) instead of by eye. Everything behavioural (physics, controls, scripts) stays in BPE2 and Lua, where it belongs.

Before you start: tool versions

Check your Blender ⇄ BRIAGE pairing check

The BRIAGE manual supplied with this project (v2.5.786) is written for **Blender 2.7x**: its material panel uses the Blender Render/Game engines and texture slots, which were removed in Blender 2.8. The Blender scene reads of the sample loco, however, show **layered actions** (“1 layer, 1 slot”), a feature of Blender 4.4 and later.

So before you begin, confirm which BRIAGE build you have and which Blender it runs in. Two workable set-ups:

- **Single Blender**: a BRIAGE build that supports your modern Blender. Material options may sit in different panels from the 2.7x manual, but the concepts in Part 2 (shader name, texture slot order, IGS options) are the same.
- **Two Blenders**: model and animate in a modern Blender, then open or append the model into a Blender 2.79 install that runs BRIAGE 2.5 and export from there. The TSC Toolkit targets Blender 4.2+; run its checks in the modelling Blender.

Folder layout you will work in

```
RailWorks\
  Source\
    MyProvider\MyProduct\
      RailVehicles\Electric\MyLoco\
        Default\Engine\ ← .igs .ia .xml
        Default\Bogies\ ← bogie .xml
        Default\CabView\ ← cab .igs .ia
        Default\Simulation\← sim .xml .csv
        Default\Engine\Textures\
        Audio\ (optional)
  Assets\
    MyProvider\MyProduct\…
      same tree, compiled (.bin .GeoPcDx …)
```

Figure 0.2 · Source and Assets mirror each other. The Kuju samples use exactly this layout (`RailVehicles\Electric\EngineSample\Default\Engine\…`). verified

Rules that save hours

- Work **inside** `Source\Provider\Product` . Save the .blend and all textures on the **same drive**; BRIAGE refuses to reference textures across drives.
- Pick your **Provider** and **Product** names once. Every blueprint reference stores them (the samples use `Developer\Addon`).
- One vehicle = one folder. Share bogies, textures and scripts between variants by pointing blueprints at the same files instead of copying them.
- Avoid dots in file names apart from the extension (BRIAGE appends `.safe` otherwise) and avoid spaces in anything a script will name.
- Keep the .blend under version control (git works well; the repo holding this guide is one).

Conventions used in every part

Convention	Meaning
Blender axes	X = right, **Y = forward**, Z = up. The front of the vehicle points along +Y.
TSC file axes	X = right, **Y = up**, **Z = forward**. BRIAGE swaps Y/Z for you on export. verified (all 33 exterior transforms match after the swap)
Blueprint 2D fields	Fields such as `FrontPivotX` , `BogeyPivotX` , `CollisionCentreX` use X = **along the track** and Y = **height**.
`1_0500_engine`	LOD level 1, visible up to 500 m, object name `engine` . See Part 2.
verified	Checked against the supplied sample files or their exports.
check	Community knowledge or inference: confirm in your own BPE2 / game build.

Glossary

Term	Meaning
Blueprint	An XML file describing one kind of object (engine, wagon, bogie, simulation, scenery…). Edited in BPE2, compiled to `.bin` .

Term	Meaning
BPE2	Blueprint Editor 2, the Kuju/DTG tool shipped with TSC for editing blueprints and exporting (compiling) them.
BRIAGE	BlendeR Intermediate Animation and Geometry Exporter: writes `.igs` (IGSE module) and `.ia` (IAE module).
.igs / .GeoPcDx	Intermediate geometry (from BRIAGE) / compiled geometry (from BPE2 export).
.ia	Intermediate animation: node positions and rotations over time, no scale.
Node / transform	A named object inside the geometry (e.g. `bo01`, `throttle`). Blueprints refer to nodes by name.
LOD	Level of detail: alternative versions of a node shown at different camera distances.
Control value	A named float on a vehicle (e.g. `Regulator`). The shared “bus” between cab, physics, script, animation and sound (Part 1).
Subsystem	The powertrain model inside an engine simulation blueprint: diesel-electric, electric or steam.
Consist	A train: an ordered list of vehicles coupled together.
serz.exe	TSC's converter between `.bin` and `.xml`. Useful for inspecting compiled files.
LogMate	TSC's log viewer. Lua `print()` output and engine messages appear here.

> **How to use this set**
>
> Read Part 1 once. It explains why every later rule exists. Then keep Parts 2–6 open as a checklist while you build, and install the toolkit from Part 7 on day one: it is cheaper to catch a naming or pivot mistake in Blender than after an export round-trip.